CSE373, MAT373: Analysis of Algorithms, Fall 2024 **Date:** October 17, 2024

Midterm Exam

(12:35 PM – 1:50 PM : 75 Minutes)

- This exam will account for 25% of your overall grade.
- There are five (5) questions, worth 75 points in total. Please answer all of them in the spaces provided.
- There are 18 pages including 9 blank pages. Please use the blank pages if you need additional space for your answers.
- This is a *closed book*, *closed notes* exam. *No cheat sheets* are allowed.
- You are *not allowed to use calculators*.

Good Luck!

Question	Pages	Parts	Points	Score
1. Recurrences	2 – 4	(a) – (d)	$4 \times 4 = 16$	
2. Asymptotic Bounds	5 – 7	(a) – (d)	$4 \times 4 = 16$	
3. An Iterative Sorting Algorithm	8 – 9	–	8	
4. Recursive Insertion Sort	10 – 14	(a) – (b)	$7 + 8 = 15$	
5. Quicksort	15 – 18	(a) – (d)	$4 \times 5 = 20$	
Total	–	–	75	

NAME: _____

SBU ID: _____

Question 1. [**16 Points**] **Recurrences.** For each of the functions below either solve it using the Master Theorem (and state the case that applies) or state why the Master Theorem does not apply. Assume that all log's are of base 2 and $T(n) = \Theta(1)$ for $n \leq 1$.

(*a*) [**4 Points**] $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + (2 + \sin n)\, n \log^3 n$

Analysis of Extended Case 2: Looking at the extended Case 2: $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant $k \geq 0$

For our problem:

- $f(n) = (2 + \sin n) n \log^3 n \in \Theta(n \log^3 n)$
- $n^{\log_b a} = n^{\log_2 2} = n^1 = n$

$f(n) = \Theta(n \log^3 n)$ can be written as: $f(n) = \Theta(n^1 \log^3 n)$

This matches the form of extended Case 2 where:

- $n^{\log_b a} = n^1$
- $k = 3$ (the power of the logarithm)

Therefore, extended Case 2 applies.

Using the formula $T(n) = \Theta(f(n) \log n)$:

$$T(n) = \Theta(n \log^3 n \cdot \log n)$$
$$T(n) = \Theta(n \log^4 n)$$

Conclusion: extended Case 2 is the correct choice here as it exactly matches our function's form $\Theta(n^{\log_b a} \log^k n)$, giving us the final complexity of $\Theta(n \log^4 n)$.

(*b*) [**4 Points**] $T(n) + T\left(\frac{n}{4}\right) = 2 \cdot T\left(\frac{n}{2}\right) + n$

Rewrite the equation as: $T(n) - T(\frac{n}{2}) = T(\frac{n}{2}) - T(\frac{n}{4}) + n$

Let $S(n) = T(n) - T(\frac{n}{2})$, so the equation becomes $S(n) = S(n/2) + n$ We can get $S(n) = \Theta(nlogn)$

Since $S(n) = T(n) - T(\frac{n}{2})$, using mater theorem: $a = 1, b = 2, f(n) = \Theta(nlogn)$

Compare $n^(0) = 1$ with $f(n) = \Theta(nlogn)$ However, the regularity condition $af\left(\frac{n}{b}\right) \leq cf(n)$ is not satisfied here because $a = 1$, and $f(n) = n \log n$ does not satisfy $f\left(\frac{n}{b}\right) \leq cf(n)$ for any $c < 1$.

Alternatively, we can use the *Recursion Tree Method* or the *Akra-Bazzi Theorem* to solve this recurrence.

Using the Recursion Tree Method:

At each level k, the cost is:

$$\text{Cost at level } k = \frac{n}{2^k} \log\left(\frac{n}{2^k}\right)$$

The total cost C is:

$$C = \sum_{k=0}^{\log_2 n} \frac{n}{2^k} \log\left(\frac{n}{2^k}\right)$$

Simplify the expression:

$$\begin{aligned} C &= \sum_{k=0}^{\log_2 n} \frac{n}{2^k} \left(\log n - \log 2^k\right) \\ &= \sum_{k=0}^{\log_2 n} \frac{n}{2^k} \left(\log n - k \log 2\right) \\ &= \sum_{k=0}^{\log_2 n} \frac{n}{2^k} \left(\log n - k\right) \end{aligned}$$

This sum evaluates to $O(n \log^2 n)$.

Therefore, the original recurrence $T(n)$ satisfies:

$$T(n) = O(n \log^2 n)$$

Answer: $T(n) = \Theta(n \log^2 n)$; that is, $T(n)$ grows proportionally to $n \log^2 n$.

(*c*) [**4 Points**] $T(n) = 4 \cdot T\left(\frac{n}{2}\right) + n^{\sin n + \cos n}$
Using Master Theorem where:
$a = 4$, $b = 2$, $f(n) = n^{\sin n + \cos n}$

Compare $n^{\log_b a} = n^{\log_2 4} = n^2$ with $f(n)$:
Since $\sin n + \cos n$ oscillates between $-\sqrt{2}$ and $\sqrt{2}$,
$n^{\sin n + \cos n}$ is always smaller than n^2

Therefore, Case 1 applies
Solution: $T(n) = \Theta(n^2)$

(*d*) [**4 Points**] $T(n) = 2^{100} \cdot T\left(\frac{n}{2}\right) + 2^n$
Using Master Theorem where:
$a = 2^{100}$, $b = 2$, $f(n) = 2^n$

Compare $n^{\log_b a} = n^{\log_2 2^{100}} = n^{100}$ with $f(n)$:
2^n grows faster than n^{100}

Therefore, Case 3 applies
Solution: $T(n) = \Theta(2^n)$

Question 2. [**16 Points**] **Asymptotic Bounds.** For each of the following statements, state whether it is *True* or *False*. Justify each answer. Assume that all log's are of base 2.

(*a*) [**4 Points**] $1 + \left(\left\lceil \frac{n}{2} \right\rceil - \left\lfloor \frac{n}{2} \right\rfloor\right) n = \Theta(n)$

Answer: **False**, The expression simplifies to:

$$1 + \left(\left\lceil \frac{n}{2} \right\rceil - \left\lfloor \frac{n}{2} \right\rfloor\right) n = \begin{cases} 1 & \text{if } n \text{ is even,} \\ 1+n & \text{if } n \text{ is odd.} \end{cases}$$

Since the function alternates between $O(1)$ and $O(n)$, it is not $\Theta(n)$.

(*b*) [**4 Points**] $n(\log n)^{100} = o\left(n^{1.000001}\right)$
Answer: **True.** We have:

$$\frac{n(\log n)^{100}}{n^{1.000001}} = n^{-0.000001}(\log n)^{100}.$$

As $n \to \infty$, $n^{-0.000001} \to 0$ and $(\log n)^{100}$ grows slowly. Therefore,

$$n^{-0.000001}(\log n)^{100} \to 0,$$

which implies $n(\log n)^{100} = o\left(n^{1.000001}\right)$.

(*c*) [**4 Points**] $T(n) = \Theta(1)$, where $T(n) = \begin{cases} \Theta\left(n^2\right) & \text{if } n \leq 2^{100}, \\ n^{\frac{1}{\log n}} & \text{otherwise.} \end{cases}$

answer: **True.** For $n \leq 2^{100}$, $T(n) = \Theta(n^2)$, but since $n \leq 2^{100}$, $T(n)$ is bounded by a constant. For $n > 2^{100}$:

$$T(n) = n^{\frac{1}{\log n}} = 2.$$

Thus, $T(n) = \Theta(1)$.

(*d*) [**4 Points**] $1 = \omega\left(\frac{1}{n}\right)$

answer: **True.** We have:

$$\lim_{n\to\infty} \frac{1}{\frac{1}{n}} = n \to \infty.$$

Therefore, $1 = \omega\left(\frac{1}{n}\right)$.

Question 3. [8 Points] An Iterative Sorting Algorithm. Show that the ITERATIVE-SORT routine given below runs in $\mathcal{O}(n \log n)$ time on an input of size n, where n is a power of 2.

```
ITERATIVE-SORT( A[ 1 : n ] )
1.  m ← 1
2.  while m < n do
3.      for j ← 1 to n/(2m) do
4.          p ← (j − 1) · 2m + 1
5.          q ← p + m − 1
6.          r ← q + m
7.          MERGE( A, p, q, r )
8.      m ← 2m
9.  return
```

Solution

Number of operations:

- Line 1: 1
- Line 2: $1+\log n$
- Line 3: $\sum_{i=1}^{\log n} 2^{i-1} + k = n + \log n - 1$
- Line 4: $\sum_{i=1}^{\log n} 2^{i-1} = n - 1$
- Line 5: $\sum_{i=1}^{\log n} 2^{i-1} = n - 1$
- Line 6: $\sum_{i=1}^{\log n} 2^{i-1} = n - 1$
- Line 7: $\sum_{i=1}^{\log n} 2^{i-1} 2^{k-i+1} = n \log n$
- Line 8: $\log n$

In total:

$$c_1 + c_2 + c_2 \log n + c_3 n + c_3 \log n - c_3 + c_4 n - c_4 + c_5 n - c_5 + c_6 - c_6 n + n \log c_7 + c_8 \log n$$

$$(c_1 + c_2 - c_3 - c_4 - c_5 - c_6) + (c_2 + c_3 + c_8) \log n + c7n \log n = \mathcal{O}(n \log n)$$

REC-MERGE(A, B, n)

Input: Two non-overlapping sorted arrays A and B containing n numbers each, where n is a power of two. **Output:** Merge A and B such that no number in A is larger than any number in B and both are sorted.

1. **if** $n = 1$ **then**
2. **if** the number in A is larger than the one in B **then** swap the two numbers
3. **else**
4. let A_L (resp. B_L) denote the left half of A (resp. B) and let A_R (resp. B_R) denote its right half
5. REC-MERGE(A_L, B_L, $\frac{n}{2}$)
6. REC-MERGE(A_R, B_R, $\frac{n}{2}$)
7. REC-MERGE(A_R, B_L, $\frac{n}{2}$)
8. **return**

REC-INSERTION-SORT(A, n)

Input: An array A containing n numbers, where n is a power of two. **Output:** The numbers in A rearranged in nondecreasing order of value.

1. **if** $n > 1$ **then**
2. let A_L denote the left half of A and let A_R denote its right half
3. REC-INSERTION-SORT(A_L, $\frac{n}{2}$)
4. REC-INSERTION-SORT(A_R, $\frac{n}{2}$)
5. REC-MERGE(A_L, A_R, $\frac{n}{2}$)
6. **return**

Figure 1: The recursive version of the insertion sort algorithm.

Question 4. [15 Points] Recursive Insertion Sort. This question asks you to analyze its running time.

(*a*) **[7 Points]** Let $T'(n)$ denote the worst-case running time of REC-MERGE while merging two arrays of size n each. What is the cost of the **divide** step of the algorithm? What is the cost of the **combine** step? Write a recurrence relation for $T'(n)$ and solve it.

The divide part of the algorithm consists of finding the midpoint of the sub-array, it costs $\mathcal{O}(1)$. The combine part of the algorithm is empty, it costs zero.

The recurrence relation looks like

$$T(n) = \begin{cases} 3T(\frac{n}{2}) + \mathcal{O}(1) & \textbf{if } n > 1 \\ \mathcal{O}(1) & \textbf{otherwise} \end{cases}$$

Comparing with the master theorem, we have, $a = 3$, $b = 2$, $f(n) = \mathcal{O}(1)$. So, $n^{\log_b a} = n^{\log_2 3}$. Thus, $f(n) = \mathcal{O}(n^{\log_2 3 - \epsilon})$, where $\epsilon = \log_2 3$.

So, case 1 of Master theorem applies. Therefore, $T(n) = \theta(n^{\log_2 3})$.

(*b*) [**8 Points**] Let $T(n)$ be the worst-case running time of REC-INSERTION-SORT on an array of size n. What is the cost of the **divide** step of the algorithm? What is the cost of the **combine** step? Write a recurrence relation for $T(n)$ and solve it.

The divide part of the algorithm consists of finding the midpoint of the sub-array, it costs $\mathcal{O}(1)$. The combine part of the algorithm is the call to REC-MERGE, which costs $\theta(n^{\log_2 3})$.

The recurrence relation looks like

$$T(n) = \begin{cases} 2T(\frac{n}{2}) + \theta(n^{\log_2 3}) & \textbf{if } n > 1 \\ \mathcal{O}(1) & \textbf{otherwise} \end{cases}$$

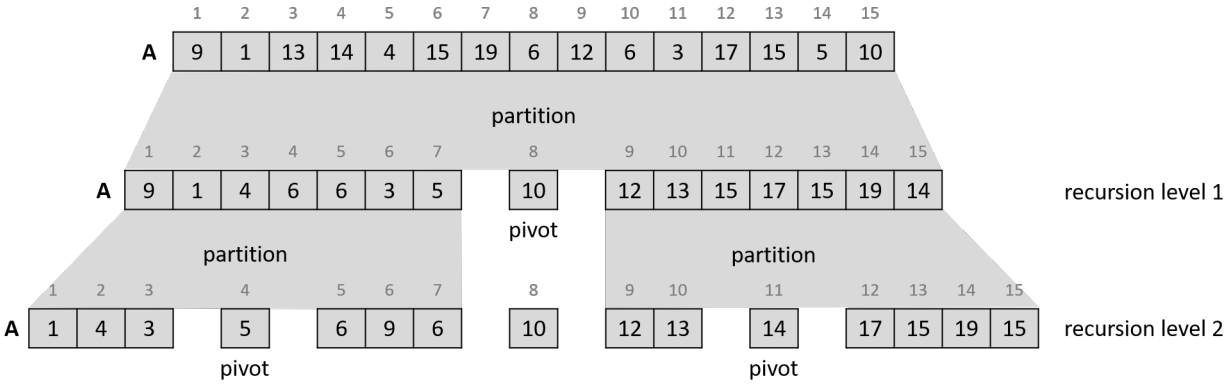
Comparing with the master theorem, we have, $a = 2$, $b = 2$, $f(n) = \theta(n^{\log_2 3})$. So, $n^{\log_b a} = n^{\log_2 2} = n$. Thus, $f(n) = \Omega(n^{1+\epsilon})$, where $\epsilon = \log_2 3 - 1 \approx 0.58$.

Furthermore, $af(\frac{n}{b}) = 2f(\frac{n}{2}) = 2(\frac{n}{2})^{\log_2 3} < n^{\log_2 3}$. So, $af(\frac{n}{b}) < kf(n)$, where $k = 1$. So, $f(n)$ satisfies the regularity condition.

So, case 3 of Master theorem applies. Therefore, $T(n) = \theta(n^{\log_2 3})$.

Question 5. [20 Points] Quicksort. Recall the in-place partitioning algorithm we learned in the class which is used by quicksort to partition the numbers in the array or subarray it is working on around the pivot. We always chose the last element of the given array/subarray as the pivot.

Given an array $A[1:15]$ as input, the following example shows the state of A after partitioning in the first two levels of recursion of quicksort.



You can describe the state of A after partitioning in recursion level 1 as:

$$[\ 9,\ 1,\ 4,\ 6,\ 6,\ 3,\ 5\ ]\ \ 10\ \ [\ 12,\ 13,\ 15,\ 17,\ 15,\ 19,\ 14\ ]$$

Then after partitioning in recursion level 2 as:

$$[\ [\ 1,\ 4,\ 3\ ]\ \ 5\ \ [\ 6,\ 9,\ 6\ ]\ ]\ \ 10\ \ [\ [\ 12,\ 13\ ]\ \ 14\ \ [\ 17,\ 15,\ 19,\ 15\ ]\ ]$$

Now, given the following array $A[1:18]$ of numbers, you will have to describe its state after partitioning at recursion levels 1, 2, 3 and 4.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
A	8	25	1	48	33	5	12	17	41	27	3	56	18	11	42	35	51	19

(*a*) **[5 Points]** Describe the state of A after partitioning at recursion level 1.

Solution:

$[8, 1, 5, 12, 17, 3, 18, 11]19[27, 25, 56, 48, 33, 42, 35, 51, 41]$

(*b*) **[5 Points]** Describe the state of A after partitioning at recursion level 2.

Solution:

$[[8, 1, 5, 3]11[12, 18, 17]]19[[27, 25, 33, 35]41[42, 48, 51, 56]]$

(*c*) **[5 Points]** Describe the state of A after partitioning at recursion level 3.

Solution:

$[[[1]3[5, 8]]11[[12]17[18]]]19[[[27, 25, 33]35]41[[42, 48, 51]56]]$

(*d*) **[5 Points]** Describe the state of A after partitioning at recursion level 4.

Solution:

[[[1]3[5, 8]]11[[12]17[18]]]19[[[[27, 25]33]35]41[[[42, 48]51]56]]